

Advanced Data Management

Column-Family NoSQL Databases

Week 6: Architecture, Data Modeling & Hands-On Lab
Bachelor of Electrical & Computer Engineering · 4-Hour Lecture

Agenda — What We Cover Today

MODULE 1— NoSQL Recap & Column-Family Fundamentals

MODULE 2— The Column-Family Data Model: Rows, Columns, Families & Partitions

MODULE 3— State-of-the-Art Systems: Cassandra, HBase, ScyllaDB & Cloud Variants

MODULE 4— CQL: Cassandra Query Language & Schema Design

MODULE 5— Replication, Consistency, Compaction & Real-World Use Cases

MODULE 6— LAB: Apache Cassandra on Linux (Docker + cqlsh)

MODULE 1

NoSQL Recap & Column-Family Fundamentals

The Four NoSQL Families

▶ KEY-VALUE STORES

- ▶ Redis, DynamoDB
- ▶ Dictionary-like: one key → one blob
- ▶ Fastest read/write; no structure

▶ DOCUMENT STORES

- ▶ MongoDB, CouchDB
- ▶ Self-contained JSON documents
- ▶ Rich queries on nested fields

▶ COLUMN-FAMILY STORES

- ▶ Cassandra, HBase, ScyllaDB
- ▶ Rows with dynamic column sets
- ▶ Best for time-series & analytics

▶ GRAPH DATABASES

- ▶ Neo4j, Amazon Neptune
- ▶ Nodes and edges
- ▶ Social networks, fraud detection

What Is a Column-Family Database?

- ▶ Stores data in rows, but each row can have a DIFFERENT set of columns — no fixed schema per row
- ▶ Columns are grouped into Column Families — physically stored together on disk
- ▶ Inspired by Google's Bigtable paper (Chang et al., 2006) — the blueprint for large-scale storage
- ▶ A keyspace contains column families; a column family contains rows; a row contains columns
- ▶ Core idea: model your data around your QUERIES, not around normalisation rules
- ▶ Optimised for: write-heavy workloads, time-series data, wide tables, analytical scans

Row-Oriented vs Column-Oriented Storage

- ▶ ROW-ORIENTED (RDBMS)
- ▶ Row 1: [id, name, email, age, score]
- ▶ Row 2: [id, name, email, age, score]
- ▶ Entire row written and read together
- ▶ Great for OLTP: fetch one complete record
- ▶ Poor for analytics: reads unused columns
- ▶ Cache is polluted with irrelevant data

- ▶ COLUMN-FAMILY (NoSQL)
- ▶ CF 'PersonalInfo': id, name, email
- ▶ CF 'Metrics': id, age, score
- ▶ Related columns stored physically together
- ▶ Scan 10M scores without touching emails
- ▶ Perfect for OLAP and time-series analytics
- ▶ Compresses similar data in same column well

Key Terminology — Glossary

- ▶ **KEYSPACE**: top-level namespace (like a 'database' in SQL); defines replication strategy
- ▶ **COLUMN FAMILY / TABLE**: a set of rows sharing related column families (like a 'table' in SQL)
- ▶ **ROW KEY / PARTITION KEY**: uniquely identifies a row; determines which node stores the data
- ▶ **COLUMN**: a key-value pair inside a row — { name: value, timestamp }; rows can have different columns
- ▶ **CLUSTERING KEY**: secondary key that sorts rows within the same partition
- ▶ **COMPACTION**: background process that merges SSTables and removes deleted/overwritten data
- ▶ **SSTable (Sorted String Table)**: immutable on-disk file format used by Cassandra and HBase

MODULE 2

The Column-Family Data Model

The Logical Data Model

- ▶ Think of a column-family table as a SORTED MAP OF SORTED MAPS
- ▶ Outer map key = Row/Partition Key → inner map = { column: value }
- ▶ Rows are sorted by partition key; columns within a row are sorted by clustering key
- ▶ Sparse rows: row A can have 3 columns, row B can have 300 — storage cost proportional to data
- ▶ Wide rows: a single partition can hold millions of column entries (e.g., all tweets of a user)
- ▶ The timestamp on each cell enables Last-Write-Wins conflict resolution automatically

Data Model: Partition → Row → Column

KEYSPACE: university_db

COLUMN FAMILY: students

PARTITION KEY: 'S001' | name: 'Maria' | dept: 'ECE' | gpa: 9.2 | year: 3

PARTITION KEY: 'S002' | name: 'Nikos' | dept: 'EE' | gpa: 8.7

PARTITION KEY: 'S003' | name: 'Anna' | gpa: 9.8 | thesis: 'FPGA Accelerators'

↑ Sparse row — no 'dept' or 'year' columns — no NULL penalty

Partition Key — The Distribution Mechanism

- ▶ The partition key is hashed using a consistent hashing function (Murmur3 in Cassandra)
- ▶ The hash value determines which node(s) in the cluster are responsible for that row
- ▶ All columns in the same partition are stored on the SAME node — efficient reads
- ▶ Query without the partition key = full cluster scan (expensive!) — always use it in WHERE
- ▶ GOOD partition key: uniformly distributes data; no single node stores disproportionate rows
- ▶ BAD partition key: low cardinality (e.g., 'country') creates hot partitions and bottlenecks

Clustering Keys – Ordering Within a Partition

- ▶ CLUSTERING KEY
- ▶ Secondary key within a partition
- ▶ Physically orders rows on disk
- ▶ Enables efficient range queries
- ▶ Example: sensor_id (partition) +
timestamp (clustering)
- ▶ Fetch last 100 readings = one seek
- ▶ No full scan needed

- ▶ PRACTICAL EXAMPLE
- ▶ Table: temperature_readings
- ▶ Partition Key: sensor_id
- ▶ Clustering Key: reading_time DESC
- ▶ `SELECT * FROM temperature_readings`
- ▶ `WHERE sensor_id = 'SENSOR_042'`
- ▶ `ORDER BY reading_time DESC LIMIT 10`

Write Path: How Data Is Written

- ▶ 1. COMMIT LOG: every write is first appended to a persistent commit log (WAL) — crash safety
- ▶ 2. MEMTABLE: write is also stored in an in-memory structure called a MemTable (fast acknowledgment)
- ▶ 3. Cassandra returns SUCCESS to the client as soon as CommitLog + MemTable are written
- ▶ 4. FLUSH: when MemTable is full, it is flushed to disk as an immutable SSTable file
- ▶ 5. COMPACTION: background process merges SSTables, removes tombstones (deletes), optimises reads
- ▶ Key insight: Cassandra NEVER updates data in-place — all writes are sequential appends
- ▶ Sequential writes are orders of magnitude faster than random writes on both HDD and SSD

Read Path: How Data Is Read

- ▶ READ REQUEST ARRIVES
 - ▶ Check MemTable first (newest data)
 - ▶ Check Row Cache (if enabled)
 - ▶ Check Bloom Filters per SSTable
 - Probabilistic: 'is key in file?'
 - Avoids reading irrelevant SSTables
 - ▶ Check Key Cache for SSTable offset
 - ▶ Read SSTable from disk if needed
- ▶ BLOOM FILTER
 - ▶ Space-efficient probabilistic structure
 - ▶ False positives possible (read an extra file)
 - ▶ False negatives IMPOSSIBLE
 - ▶ Default: 1% false positive rate in Cassandra
 - ▶ Dramatically reduces disk I/O
 - ▶ Result: merge data from all sources
 - ▶ Return most recent version to client

MODULE 3

State-of-the-Art Column-Family Systems

Apache Cassandra — Overview

▶ ORIGIN

- ▶ Created at Facebook (2007-2008)
- ▶ Released open-source 2008
- ▶ Apache top-level project since 2010
- ▶ Current: Cassandra 5.x (2024)

▶ ARCHITECTURE

- ▶ Peer-to-peer — NO single master
- ▶ Every node is equal (ring topology)
- ▶ Gossip protocol for node discovery
- ▶ Consistent hashing for data placement

▶ KEY STRENGTHS

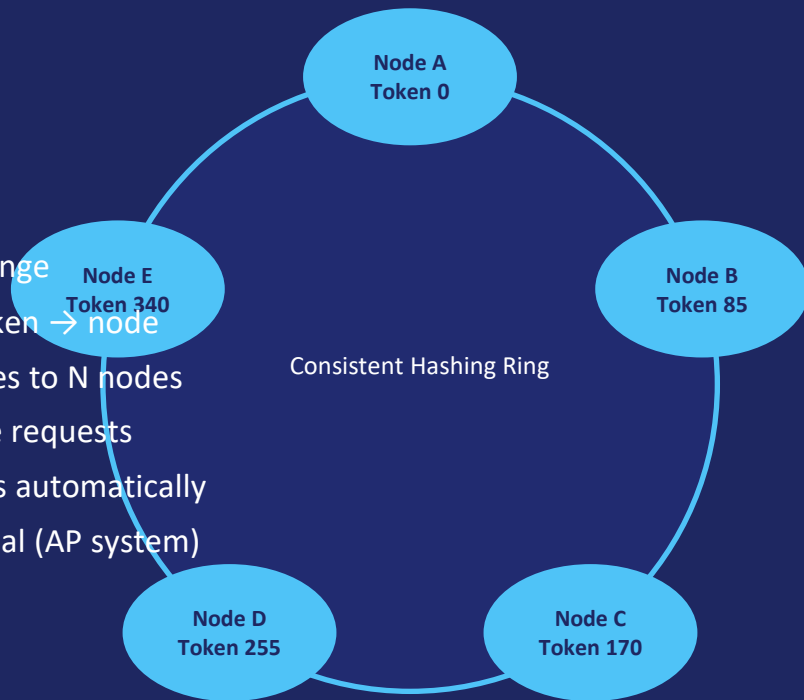
- ▶ Linear horizontal scalability
- ▶ Multi-datacenter replication built-in
- ▶ Tunable consistency (ONE to ALL)
- ▶ CQL — SQL-like query language
- ▶ Excellent write throughput

▶ REAL-WORLD USERS

- ▶ Apple: 75,000+ nodes, 10PB+ data
- ▶ Netflix: global streaming metadata
- ▶ Discord: trillions of messages stored

Cassandra Ring Architecture

- ▶ Each node owns a token range
- ▶ Partition key → hash → token → node
- ▶ Replication: each row copies to N nodes
- ▶ Node failure: replicas serve requests
- ▶ New node joins: rebalances automatically
- ▶ No master — all nodes equal (AP system)



Cassandra Consistency Levels — Tunable CAP

- ▶ Cassandra lets you choose consistency per read AND per write operation — unique flexibility
- ▶ ONE: fastest; returns after 1 replica responds — highest availability, lowest consistency
- ▶ QUORUM: majority of replicas must agree — balanced; ensures no stale read after a write
- ▶ ALL: every replica must respond — strongest consistency, lowest availability
- ▶ LOCAL_QUORUM: quorum within the local datacenter only — ideal for multi-DC deployments
- ▶ Rule of thumb: if $\text{write_CL} + \text{read_CL} > \text{replication_factor}$ → STRONG CONSISTENCY guaranteed
- ▶ Example: RF=3, write=QUORUM(2), read=QUORUM(2) → $2+2>3$ → always reads latest write

Apache HBase — Overview

▶ ORIGIN

- ▶ Based on Google Bigtable paper (2006)
- ▶ Part of Apache Hadoop ecosystem
- ▶ First release: 2007; v2.x currently
- ▶ Tight integration with HDFS and MapReduce

▶ ARCHITECTURE

- ▶ Master-RegionServer model
- ▶ HMaster: coordinates, manages metadata
- ▶ RegionServers: serve read/write requests
- ▶ ZooKeeper: distributed coordination

▶ KEY STRENGTHS

- ▶ Strong consistency (CP system)
- ▶ Co-location with HDFS data
- ▶ Efficient range scans over sorted keys
- ▶ Billions of rows × millions of columns
- ▶ Versioned cells: keep N versions per cell

▶ REAL-WORLD USERS

- ▶ Facebook Messenger (originally)
- ▶ Yahoo! — hundreds of PB
- ▶ Adobe, Twitter/X analytics

ScyllaDB – The C++ Replacement

▶ ORIGIN

- ▶ Founded 2014 by Avi Kivity (KVM creator)
- ▶ C++20 rewrite of Apache Cassandra
- ▶ Cassandra-compatible: same CQL, SSTables
- ▶ Open source; ScyllaDB Cloud (DBaaS)
- ▶ Latest: ScyllaDB 6.x (2024-2025)

▶ TECHNOLOGY

- ▶ Seastar async framework (no threads)
- ▶ Shard-per-core: each CPU core owns its data
- ▶ Zero-copy I/O, huge pages, DPDK networking

▶ PERFORMANCE CLAIMS

- ▶ Up to 2M requests/sec on 1 machine
- ▶ 10× Cassandra throughput per node
- ▶ Consistent sub-millisecond p99 latency
- ▶ Samsung benchmark: 10–37× speedup
- ▶ Independent tests: 2–5× typical

▶ REAL-WORLD USERS

- ▶ Discord: switched from Cassandra 2023
- ▶ Grab (ride-sharing SEA)
- ▶ AppNexus, Strava, Zillow

Cloud-Native Column-Family Services

- ▶ AMAZON KEYSPACES
 - ▶ Managed Cassandra-compatible service
 - ▶ Serverless: auto-scales capacity
 - ▶ No cluster management needed
 - ▶ CQL-compatible — easy migration
 - ▶ Strong consistency mode available
- ▶ GOOGLE BIGTABLE
 - ▶ The original inspiration for all CFDBs
 - ▶ Managed, fully serverless
 - ▶ Powers Gmail, Google Search, Maps
- ▶ AZURE COSMOS DB (Cassandra API)
 - ▶ Multi-model: also JSON, Gremlin, Table
 - ▶ Global distribution in minutes
 - ▶ 99.999% SLA
- ▶ DATASTAX ASTRA DB
 - ▶ Managed Cassandra on any cloud
 - ▶ Vector search support (2024)
 - ▶ REST and GraphQL APIs built-in
- ▶ TREND: all major clouds now offer managed column-family as a service

System Comparison at a Glance

System	Consistency	Language	Architecture	Best For
Apache Cassandra	Tunable (AP)	Java	Peer-to-peer ring	High availability, geo-distributed
Apache HBase	Strong (CP)	Java	Master-RegionServer	Hadoop analytics, range scans
ScyllaDB	Tunable (AP)	C++20	Shard-per-core ring	Ultra-low latency, IoT, finance
Google Bigtable	Strong (CP)	Managed	Proprietary cloud	GCP analytics, petabyte scale
Amazon Keyspaces	Tunable	Managed	Serverless	AWS-native, CQL migration

MODULE 4

CQL — Cassandra Query Language & Schema Design

CQL — Cassandra Query Language

- ▶ CQL looks like SQL — intentionally designed to minimise the learning curve for developers
- ▶ Key difference: CQL enforces query-driven design — only queries using partition key are efficient
- ▶ No JOIN operations — denormalisation is required and expected
- ▶ No subqueries — all complexity is modelled into the table structure at design time
- ▶ No arbitrary WHERE clauses — columns used in WHERE must be part of the primary key
- ▶ Operations: CREATE KEYSPACE, CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, BATCH
- ▶ Tool: cqlsh — the interactive command-line shell (used in today's lab)

CQL — Keyspace & Table Creation

```
-- Create a keyspace with replication strategy
CREATE KEYSPACE university
  WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'datacenter1': 3
  } AND durable_writes = true;

USE university;

-- Create a table (column family)
CREATE TABLE students (
  student_id    TEXT,
  enrolled_year INT,
  name          TEXT,
  department    TEXT,
  gpa           FLOAT,
  PRIMARY KEY ((student_id), enrolled_year)
) WITH CLUSTERING ORDER BY (enrolled_year DESC);
```

CQL – Insert, Select & Filtering

```
-- Insert data
```

```
INSERT INTO students (student_id, enrolled_year, name, department, gpa)  
VALUES ('S001', 2022, 'Maria Papadopoulou', 'ECE', 9.2);
```

```
INSERT INTO students (student_id, enrolled_year, name, department, gpa)  
VALUES ('S002', 2023, 'Nikos Georgiou', 'EE', 8.7);
```

```
-- Select all rows for a student (uses partition key -- fast)
```

```
SELECT * FROM students WHERE student_id = 'S001';
```

```
-- Range query using clustering key
```

```
SELECT * FROM students  
WHERE student_id = 'S001' AND enrolled_year >= 2022  
LIMIT 10;
```

```
-- WARNING: this triggers ALLOW FILTERING (full scan -- avoid in production)
```

```
SELECT * FROM students WHERE gpa > 9.0 ALLOW FILTERING;
```

Query-Driven Design Principle

▶ RELATIONAL DESIGN

- ▶ 1. Identify your entities
 - ▶ 2. Normalise to 3NF
 - ▶ 3. Add indexes for performance
 - ▶ 4. Use JOINS to answer queries
-
- ▶ You design the SCHEMA first,
 - ▶ then figure out the queries
-
- ▶ Works for OLTP; poor at scale

▶ CASSANDRA DESIGN

- ▶ 1. List all application queries
 - ▶ 2. Design ONE TABLE PER QUERY
 - ▶ 3. Partition key = equality filter
 - ▶ 4. Clustering key = range/sort filter
-
- ▶ You design the QUERIES first,
 - ▶ then design the tables to serve them
-
- ▶ Denormalisation = acceptable trade-off

Data Modelling Example — IoT Sensor Data

```
-- QUERY: Get last N readings for a specific sensor
-- Primary key: (sensor_id) + reading_time DESC
```

```
CREATE TABLE sensor_readings (
  sensor_id      TEXT,
  reading_time   TIMESTAMP,
  temperature    FLOAT,
  humidity       FLOAT,
  pressure       FLOAT,
  PRIMARY KEY ((sensor_id), reading_time)
) WITH CLUSTERING ORDER BY (reading_time DESC);
```

```
-- Insert a reading
INSERT INTO sensor_readings (sensor_id, reading_time, temperature, humidity)
VALUES ('SENSOR_042', toTimestamp(now()), 23.4, 61.2);
```

```
-- Get last 5 readings -- very fast: one partition read
SELECT * FROM sensor_readings WHERE sensor_id = 'SENSOR_042' LIMIT 5;
```

Secondary Indexes & Materialized Views

```
-- Secondary index: allows queries on non-key columns
-- WARNING: high cardinality columns cause performance issues
CREATE INDEX ON students (department);
```

```
-- Now this query works (but still does a scatter read):
SELECT * FROM students WHERE department = 'ECE';
```

```
-- BETTER: Use a Materialized View or a dedicated table
-- Cassandra 5.x supports Materialized Views (with caveats)
CREATE MATERIALIZED VIEW students_by_dept AS
  SELECT * FROM students
  WHERE department IS NOT NULL AND student_id IS NOT NULL
  PRIMARY KEY (department, student_id);
```

```
-- Now query efficiently by department:
SELECT * FROM students_by_dept WHERE department = 'ECE';
```

CQL Data Types — Key Reference

- ▶ TEXT / VARCHAR: UTF-8 string — use for IDs, names, labels
- ▶ UUID / TIMEUUID: universally unique ID — TIMEUUID embeds a timestamp, useful as clustering key
- ▶ TIMESTAMP: date + time with timezone — always store in UTC
- ▶ INT / BIGINT / FLOAT / DOUBLE: numeric types — use BIGINT for counters and epoch milliseconds
- ▶ BOOLEAN: true / false — stored as single byte
- ▶ LIST<T>: ordered collection — ['a', 'b', 'c']
- ▶ SET<T>: unordered unique values — { 'redis', 'cassandra' }
- ▶ MAP<K,V>: key-value pairs — { 'course': 'ADM', 'grade': '9.0' }
- ▶ COUNTER: special type for distributed atomic increments — cannot mix with other columns

MODULE 5

Replication, Consistency, Compaction & Real-World

Replication Strategies

- ▶ SimpleStrategy: places replicas on the next N nodes clockwise in the ring — single DC only
- ▶ NetworkTopologyStrategy: places replicas per datacenter — REQUIRED for multi-DC deployments
- ▶ Replication Factor (RF): how many copies of each row exist across the cluster
- ▶ RF=1: no redundancy — single node failure loses data permanently
- ▶ RF=3: industry standard — tolerates 1 node failure with QUORUM reads/writes
- ▶ RF=5: used by Apple, Netflix for 2-node-failure tolerance at global scale
- ▶ Higher RF increases storage cost but improves availability and read throughput (more nodes can serve)

CAP Theorem & Column-Family Placement

▶ WHERE DO SYSTEMS FALL?

- ▶ CAP THEOREM (Brewer 2000)
- ▶ A distributed system can guarantee
- ▶ at most TWO of three properties:
- ▶ C = Consistency
 - ▶ Every read gets the latest write
- ▶ A = Availability
 - ▶ Every request gets a response
- ▶ P = Partition Tolerance
 - ▶ System works despite network splits
- ▶ P is always required in practice →
- ▶ choose between C and A

- ▶ CASSANDRA / SCYLLADB: AP system
 - ▶ Tunable: can lean toward C
 - ▶ Default: availability-first
- ▶ HBASE / GOOGLE BIGTABLE: CP system
 - ▶ Consistency guaranteed
 - ▶ May reject writes during partition
- ▶ AMAZON KEYSPACES: tunable
 - ▶ Configurable per table
- ▶ Real world: PACELC model is more
- ▶ nuanced — consider latency too

Compaction Strategies

- ▶ Cassandra never deletes or updates in-place — writes always create new SSTables on disk
- ▶ Compaction: background merging of SSTables — removes old versions and tombstones (deletes)
- ▶ STCS (Size-Tiered Compaction Strategy): merges similarly-sized SSTables — default; write-optimised
- ▶ LCS (Leveled Compaction Strategy): maintains fixed-size levels — read-optimised; more I/O
- ▶ TWCS (Time-Window Compaction Strategy): compacts within time windows — ideal for time-series
- ▶ TWCS is the best choice for IoT/metrics tables where data becomes immutable after a time window
- ▶ ScyllaDB ICS (Incremental Compaction Strategy): lower write amplification than all Cassandra strategies

Real-World Use Cases — Engineering Context

- ▶ NETFLIX: stores viewing history, recommendations metadata — 15M+ operations/sec globally
- ▶ DISCORD: switched from Cassandra to ScyllaDB (2023) — serves trillions of messages, sub-ms latency
- ▶ APPLE iCloud: 75,000+ Cassandra nodes — stores contacts, calendar, notes for 1B+ users
- ▶ IoT PLATFORMS: time-series sensor data — temperature, pressure, power consumption at millisecond intervals
- ▶ TELECOMMUNICATIONS: call detail records (CDRs) — billions of call events per day, legal retention requirements
- ▶ FINANCIAL SERVICES: trade tick data, market feed storage — microsecond timestamps, immutable records
- ▶ HEALTHCARE: patient sensor streams (ICU monitors) — high-frequency writes, time-ordered queries

Case Study: Discord's Migration to ScyllaDB

BEFORE (Cassandra)

- ▶ Hot partitions causing latency spikes
- ▶ JVM GC pauses affecting p99 latency
- ▶ Trillions of messages with growing backlog
- ▶ Compaction pressure on large partitions
- ▶ Operational complexity at JVM level
- ▶ p99 latency: 40-125ms (unacceptable)

AFTER (ScyllaDB)

- ▶ Shard-per-core eliminates hot spots
- ▶ C++ has no GC pauses
- ▶ Same data model and CQL queries
- ▶ ScyllaDB ICS handles large partitions
- ▶ Dramatically simpler JVM-free ops
- ▶ p99 latency: < 15ms consistently

MODULE 6 — LAB

Apache Cassandra on Linux: Docker + cqlsh

Lab Overview & Objectives

- ▶ GOAL: Run Apache Cassandra in Docker on Linux and interact via cqlsh
- ▶ By the end of the lab you will be able to: start a Cassandra container, connect with cqlsh, create a keyspace and table, perform INSERT / SELECT / UPDATE / DELETE, and model a real schema
- ▶ PREREQUISITES: Docker installed and running; internet access for pulling the image
- ▶ Estimated time: 60 minutes (Steps 1-11)
- ▶ All commands are for Ubuntu/Debian Linux — should also work on WSL2 on Windows
- ▶ No Python or Java required — we use cqlsh inside the Docker container

Step 1 – Install Docker (if needed)

```
# Update package list
```

```
sudo apt update
```

```
# Install Docker
```

```
sudo apt install -y docker.io
```

```
# Start Docker service
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
# Add your user to the docker group (avoid sudo every time)
```

```
sudo usermod -aG docker $USER
```

```
# Verify installation
```

```
docker --version
```

```
# Expected output: Docker version 24.x.x or later
```

```
# NOTE: log out and back in after usermod for group change to take effect
```

Step 2 — Pull & Run Cassandra Container

```
# Pull the official Cassandra image (latest LTS)
```

```
docker pull cassandra:4.1
```

```
# Run Cassandra container
```

```
# -d: detached (background)
```

```
# -p: expose CQL native port 9042
```

```
# --name: container alias for easy reference
```

```
docker run -d \  
  --name cassandra-lab \  
  -p 9042:9042 \  
  cassandra:4.1
```

```
# Wait ~30 seconds for Cassandra to fully start
```

```
# Check status (look for 'Up' in STATUS column)
```

```
docker ps
```

```
# Optionally check Cassandra startup logs
```

```
docker logs cassandra-lab | tail -20
```


Step 3 – Connect with cqlsh

```
# Start cqlsh inside the running container
# cqlsh is the Cassandra Query Language shell
docker exec -it cassandra-lab cqlsh

# You should see:
# Connected to Test Cluster at 127.0.0.1:9042
# [cqlsh 6.1.0 | Cassandra 4.1.x | CQL spec 3.4.6]
# Use HELP for help.
# cqlsh>

# Explore: list all keyspaces
DESCRIBE KEYSPACES;

# Exit cqlsh
EXIT;

# If cqlsh fails with connection error, wait and retry:
# docker exec -it cassandra-lab cqlsh --connect-timeout=30
```

Step 4 — Create Keyspace

```
-- Inside cqlsh:

-- Create a keyspace for our university application
-- SimpleStrategy is fine for a single-node development cluster
CREATE KEYSPACE IF NOT EXISTS university
  WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
  };

-- Switch to the keyspace
USE university;

-- Confirm we are in the right keyspace
-- Prompt should change to: cqlsh:university>

DESCRIBE KEYSPACE university;

-- You will see the full CQL definition of the keyspace
```

Step 5 – Create Table for Student Records

```
-- Create a students table
-- Partition key: student_id (equality lookup by student)
-- Clustering key: enrolled_year DESC (latest year first)
CREATE TABLE IF NOT EXISTS students (
  student_id    TEXT,
  enrolled_year INT,
  name          TEXT,
  department    TEXT,
  gpa           FLOAT,
  courses       SET<TEXT>,
  PRIMARY KEY ((student_id), enrolled_year)
) WITH CLUSTERING ORDER BY (enrolled_year DESC);

-- Verify table was created
DESCRIBE TABLE students;

-- Check all tables in keyspace
DESCRIBE TABLES;
```

Step 6 – Insert Student Data

```
-- Insert several student records
INSERT INTO students (student_id, enrolled_year, name, department, gpa, courses)
VALUES ('S001', 2022, 'Maria Papadopoulou', 'ECE', 9.2,
       {'Advanced_Data_Management', 'Digital_Signal_Processing', 'VLSI_Design'});

INSERT INTO students (student_id, enrolled_year, name, department, gpa, courses)
VALUES ('S002', 2023, 'Nikos Georgiou', 'EE', 8.7,
       {'Advanced_Data_Management', 'Power_Systems'});

INSERT INTO students (student_id, enrolled_year, name, department, gpa, courses)
VALUES ('S003', 2021, 'Anna Stavridou', 'ECE', 9.8,
       {'Advanced_Data_Management', 'Machine_Learning', 'Embedded_Systems'});

-- Verify inserts
SELECT * FROM students;
```

Step 7 – Query & Filter Data

```
-- Efficient query: uses partition key
SELECT * FROM students WHERE student_id = 'S001';

-- Range query using clustering key
SELECT name, gpa FROM students
  WHERE student_id = 'S001'
  AND enrolled_year >= 2021;

-- Count rows in partition
SELECT COUNT(*) FROM students WHERE student_id = 'S001';

-- Query with TTL: data auto-expires after 86400 seconds (1 day)
INSERT INTO students (student_id, enrolled_year, name, department, gpa)
VALUES ('TEMP01', 2024, 'Temp Student', 'ECE', 0.0)
USING TTL 86400;

-- Check TTL remaining
SELECT TTL(name) FROM students WHERE student_id = 'TEMP01';
```

Step 8 — Update & Delete

-- Update GPA for a student

```
UPDATE students SET gpa = 9.5
WHERE student_id = 'S001' AND enrolled_year = 2022;
```

-- Add a course to the SET collection

```
UPDATE students
SET courses = courses + {'Wireless_Communications'}
WHERE student_id = 'S001' AND enrolled_year = 2022;
```

-- Remove a course from the SET

```
UPDATE students
SET courses = courses - {'VLSI_Design'}
WHERE student_id = 'S001' AND enrolled_year = 2022;
```

-- Delete a single column value

```
DELETE gpa FROM students WHERE student_id = 'S002' AND enrolled_year = 2023;
```

-- Delete an entire row

```
DELETE FROM students WHERE student_id = 'TEMP01' AND enrolled_year = 2024;
```

-- Verify changes

```
SELECT * FROM students WHERE student_id = 'S001';
```

Step 9 — IoT Sensor Table (Bonus Task)

```
-- Create a time-series table for sensor readings
-- This is the most common Cassandra use case in ECE/EE
CREATE TABLE IF NOT EXISTS sensor_readings (
    sensor_id    TEXT,
    reading_time  TIMESTAMP,
    temperature   FLOAT,
    humidity      FLOAT,
    voltage       FLOAT,
    PRIMARY KEY ((sensor_id), reading_time)
) WITH CLUSTERING ORDER BY (reading_time DESC)
    AND default_time_to_live = 2592000; -- 30 days TTL

-- Insert several readings
INSERT INTO sensor_readings (sensor_id, reading_time, temperature, humidity, voltage)
VALUES ('SENSOR_ECE_01', '2025-04-25 10:00:00+0000', 23.4, 61.2, 220.1);

INSERT INTO sensor_readings (sensor_id, reading_time, temperature, humidity, voltage)
VALUES ('SENSOR_ECE_01', '2025-04-25 10:01:00+0000', 23.6, 61.0, 219.9);

-- Get last 10 readings for sensor
SELECT * FROM sensor_readings WHERE sensor_id = 'SENSOR_ECE_01' LIMIT 10;
```

Step 10 – Inspect Cluster & Cleanup

```
-- Check cluster information from cqlsh
```

```
SELECT cluster_name, data_center, rack FROM system.local;
```

```
-- Check all nodes in the cluster (just one for dev)
```

```
SELECT peer, data_center, rack, release_version  
FROM system.peers;
```

```
-- Check token ranges and ownership
```

```
-- (in a multi-node cluster this shows distribution)
```

```
SELECT tokens FROM system.local;
```

```
-- Drop the keyspace (cleanup)
```

```
DROP KEYSPACE IF EXISTS university;
```

```
-- Exit cqlsh
```

```
EXIT;
```

```
# Back in the Linux terminal: stop and remove the container
```

```
docker stop cassandra-lab
```

```
docker rm cassandra-lab
```

```
# Verify container is removed
```

```
docker ps -a
```


Step 11 – Advanced: nodetool (Cluster Management)

```
# nodetool: the Cassandra administration command-line tool
```

```
# Run it inside the container
```

```
# Check cluster status -- shows node health, load, token ownership
```

```
docker exec cassandra-lab nodetool status
```

```
# Expected output (single node):
```

```
# Datacenter: datacenter1
```

```
# Status=Up/Down, State=Normal/Leaving/Joining/Moving
```

#	Address	Load	Tokens	Owns	Host ID	Rack
#	UN 127.0.0.1	100KB	256	100%	<uuid>	rack1

```
# Show ring token assignments
```

```
docker exec cassandra-lab nodetool ring
```

```
# Check compaction statistics
```

```
docker exec cassandra-lab nodetool compactionstats
```

```
# Trigger manual compaction on keyspace
```

```
docker exec cassandra-lab nodetool compact university
```

Common Errors & Troubleshooting

- ▶ `cqlsh: NoHostAvailable` — Cassandra not ready yet; wait 30s and retry or check docker logs
- ▶ Cannot execute this query as it might involve data filtering — add `ALLOW FILTERING` or redesign schema
- ▶ `InvalidRequest: Non PRIMARY KEY columns found in where clause` — you cannot filter on a non-key column
- ▶ `write timeout` — RF mismatch with consistency level; for single node always use `CL ONE`
- ▶ `Column family 'x' does not exist` — forgot to `USE keyspace`; run `USE university`;
- ▶ `docker: Error response from daemon: port is already allocated` — port 9042 in use; change `-p` flag
- ▶ `Compaction is paused due to tombstones threshold` — too many deletes; run `nodetool compact`

Summary

Key Takeaways from Today's Lecture

What We Learned Today

- ▶ Column-family databases store data in sparse rows with dynamic column sets — inspired by Google Bigtable
- ▶ The data model: Keyspace → Column Family → Partition Key → Clustering Key → Columns
- ▶ Write path: CommitLog + MemTable → SSTable (append-only, sequential writes = speed)
- ▶ Partition key determines data placement; clustering key determines on-disk sort order
- ▶ Apache Cassandra: peer-to-peer ring, tunable consistency, industry gold standard
- ▶ Apache HBase: strong consistency, Hadoop-integrated, CP system with master architecture
- ▶ ScyllaDB: C++20 rewrite, shard-per-core, dramatically lower latency than Java Cassandra
- ▶ CQL enforces query-driven design — model tables around query patterns, not normalisation
- ▶ Real-world: Netflix, Apple, Discord, IoT platforms all rely on column-family databases

Next Steps & Further Reading

▶ REQUIRED READING

- ▶ Chang et al. (2006) — Bigtable: A Distributed Storage System for Structured Data (Google)
- ▶ Discord Engineering Blog — Cassandra to ScyllaDB migration (2023)
- ▶ Cassandra Data Modeling (datastax.com/learn)

▶ TOOLS TO EXPLORE

- ▶ DataStax Studio — GUI for Cassandra
- ▶ Apache Cassandra Docs — cassandra.apache.org
- ▶ ScyllaDB University (free online)
- ▶ DataStax Free Tier (Astra DB cloud)

▶ WEEK 7 PREVIEW

- ▶ Graph Databases: Neo4j and network data
- ▶ Difference between property graphs and RDF
- ▶ Cypher query language
- ▶ Use cases: fraud detection, social networks, knowledge graphs

▶ EXAM TOPICS

- ▶ Design a CQL schema for a given use case
- ▶ Compare Cassandra vs HBase trade-offs
- ▶ Explain the write path and compaction
- ▶ Calculate consistency given RF and CL

Thank You

Questions & Discussion

Advanced Data Management — Week 6
Bachelor of Electrical & Computer Engineering